

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

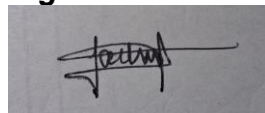
QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I J a s h w a n t h . G . R (1 9 2 5 2 4 3 8 4) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Sequential File Organization vs Heap File Organization

Both are basic file organizations for storing database records; they differ fundamentally in how records are physically arranged, which drives very different costs for insertion, deletion, and retrieval.

Aspect	Sequential File Organization	Heap (Unordered) File Organization
Physical order	Records stored in sorted order of a key field	Records stored in no particular order (insertion order)
Insertion	Slow — $O(n)$: correct position must be located and subsequent records shifted (or an overflow area is used, which needs periodic reorganization)	Fast — $O(1)$: new record simply appended at the end of the file
Deletion	Slow — $O(n)$: record located by search, then remaining records shifted to close the gap (or marked deleted, requiring later compaction)	Fast — $O(1)$ to mark, but reclaiming the space still needs an $O(n)$ compaction pass eventually
Retrieval (exact match)	Fast — $O(\log n)$ using binary search since the file is ordered	Slow — $O(n)$: every block may need to be scanned on average $n/2$ blocks
Retrieval (range query)	Fast — $O(\log n + k)$: binary search to the start, then sequential scan of k matching records	Slow — $O(n)$: entire file must be scanned since no ordering exists

Analysis

Sequential organization trades insertion/deletion speed for search speed: maintaining sort order accelerates both exact-match and range retrieval, but every update pays a reorganization cost. Heap organization does the opposite: it is optimal when records are added far more often than they are searched, since appends are $O(1)$, but any search degrades to a full linear scan. In practice, sequential files suit read-heavy, relatively static data (e.g., archived records, reports), while heap files suit write-heavy, transient staging areas where data is later indexed or loaded into a more structured organization.

Q2. Clustered Indexing vs Non-Clustered Indexing

Aspect	Clustered Index	Non-Clustered Index
Physical data order	Data rows are physically sorted/stored in the same order as the index key	Data rows are stored independently; the index holds a separate structure of (key, pointer) pairs
Number per table	Only one per table (since data can be physically sorted only one way)	Multiple non-clustered indexes can exist on the same table
Storage overhead	Low — no separate copy of data, index is the data's physical order	Higher — a full separate index structure with pointers back to the data rows
Retrieval speed (on indexed column)	Very fast — data found directly at the sorted location, ideal for range queries	Fast, but one extra hop is needed from the index entry to the actual data row (bookmark lookup)
Example	A B+ Tree clustering index on Employee.EmployeeID, where the employee table itself is physically ordered by EmployeeID	A B+ Tree non-clustering index on Employee.Department in the same table, used to quickly find employees by department without reordering the table

Analysis

Because clustered indexing determines the physical layout of the table, it gives the best performance for range scans and is naturally built on the primary key. Non-clustered indexes provide additional fast access paths on other frequently queried columns without disturbing the primary physical order, at the cost of extra storage and an indirection step to reach the actual row. A well-designed schema typically has exactly one clustered index (usually the primary key) supplemented by several non-clustered indexes on columns used in WHERE/JOIN clauses.

Q3. Primary Indexing vs Secondary Indexing

Aspect	Primary Index	Secondary Index
Built on	The primary key of an ordered (sequentially organized) file	Any non-primary-key attribute, key or non-key, unique or duplicate-valued
Index type	Usually sparse — one entry per block, since the file is already sorted by the key	Usually dense — one entry per distinct value (or per record if unique), since the file is not physically ordered by this attribute
Storage requirement	Low — sparse structure needs far fewer entries than there are records	Higher — dense structure needs an entry for every searchable value, and duplicate values need pointer buckets
Search performance	Very fast — combines a small sparse index lookup with a short sequential scan within the target block	Slightly slower than primary indexing per lookup — dense index is larger and duplicate-key buckets add an extra pointer-chasing step, but still $O(\log n)$ overall
Number allowed	One per file (file can be sorted only one way)	Many per file — one for each attribute that needs fast lookup

Analysis

Primary indexing is inherently more storage-efficient because it piggybacks on the file's own physical order and needs only sparse entries, whereas secondary indexing must be dense (or use pointer buckets for duplicates) because the underlying file is not ordered by that attribute. The trade-off is flexibility: secondary indexes let queries efficiently target any attribute, not just the primary key, at the cost of additional storage and slightly higher per-lookup overhead.

Q4. Static Hashing vs Dynamic Hashing

Aspect	Static Hashing	Dynamic Hashing (e.g., Extendible / Linear)
Number of buckets	Fixed at creation time, does not change	Grows and shrinks automatically as the file grows/shrinks
Handling growth	Overflow chaining once buckets fill; eventually needs a full, expensive rehash/reorganization	Buckets split or the directory expands incrementally — no full-file reorganization needed
Performance under growth	Degrades toward $O(n)$ as overflow chains lengthen if the file grows beyond the planned size	Remains close to $O(1)$ even as the file grows, since only local restructuring occurs
Implementation complexity	Simple — a fixed hash function and bucket array	More complex — needs a directory (extendible hashing) or a controlled bucket-split order (linear hashing)
Best suited for	Files with a known, stable size (e.g., a fixed lookup table)	Files with unpredictable or continuous growth (e.g., active transactional systems)

Analysis

Static hashing is simple and gives $O(1)$ access as long as the file size stays close to its planned capacity, but it degrades badly and requires costly reorganization once the data outgrows the original bucket count. Dynamic hashing techniques solve exactly this problem by allowing the bucket structure to grow incrementally, keeping access near $O(1)$ indefinitely at the cost of added implementation complexity (directory maintenance or controlled split pointers). Systems with volatile, hard-to-predict growth should prefer dynamic hashing; systems with stable, well-understood data volumes can use the simpler static scheme.

Q5. B-Tree Indexing vs B+ Tree Indexing for Large-Scale Applications

Aspect	B-Tree	B+ Tree
Data pointers	Stored in both internal and leaf nodes	Stored only in leaf nodes; internal nodes hold keys only (used purely as a guide)
Fan-out per node	Lower, since internal nodes also carry data pointers	Higher — more keys fit per internal node, so the tree is shorter for the same data volume
Tree height (large-scale data)	Taller for the same number of records → more disk I/Os per search	Shorter → fewer disk I/Os per search, which matters greatly at large scale
Range queries / sequential access	Requires repeated tree traversal, since leaves are not linked	Very fast — leaf nodes are linked in a sequential chain, ideal for range scans
Insertion / deletion complexity	More complex — must manage data pointers that may exist at any level	Simpler — restructuring is largely confined to the leaf level

Analysis

For large-scale database applications, the B+ Tree is generally preferred. Its higher fan-out keeps the tree shallow even for millions of records, directly reducing the number of disk block accesses per search. Its linked-leaf structure makes range queries and sequential scans — extremely common in analytical and reporting workloads — far more efficient than a B-Tree, which must re-traverse from the root for each range boundary. This is why virtually all production RDBMS engines (e.g., MySQL InnoDB, PostgreSQL, Oracle) implement B+ Trees rather than classical B-Trees for their primary indexing structure.

Q6. Linear Hashing vs Extendible Hashing

Aspect	Linear Hashing	Extendible Hashing
Directory structure	No directory — buckets are split in a predetermined linear order using a split pointer	Uses a directory of 2^d pointers (d = global depth) that maps hash prefixes to buckets

Aspect	Linear Hashing	Extendible Hashing
Bucket splitting	Splits the bucket pointed to by the split pointer, regardless of which bucket actually overflowed (may use temporary overflow chaining)	Splits exactly the bucket that overflowed; if its local depth reaches the global depth, the directory doubles
Scalability	Scales smoothly with no directory-doubling cost, but overflow chains can temporarily appear before the split pointer reaches the overflowing bucket	Scales well but directory doubling is an all-at-once (though infrequent) operation whose cost grows with directory size
Extra storage overhead	No directory overhead — only the bucket array plus occasional overflow blocks	Directory adds an extra level of indirection and memory overhead, especially for large global depth
Access cost	1–2 block accesses typically (bucket access, occasional overflow chain)	Directory access (often memory-resident) + 1 bucket access — typically very close to $O(1)$

Analysis

Both are dynamic hashing schemes that avoid full-file reorganization, but they manage growth differently. Linear hashing avoids directory overhead entirely and splits buckets in a fixed round-robin order, which is simple and memory-efficient but can leave some buckets with longer overflow chains temporarily. Extendible hashing always splits the actual overflowing bucket, keeping access times more uniformly low, but pays for this with directory storage and periodic directory-doubling cost. Linear hashing is preferred when memory for a directory is scarce and mild, temporary overflow is acceptable; extendible hashing is preferred when consistently low, predictable access time is more important than directory overhead.

Q7. Dense Indexing vs Sparse Indexing

Aspect	Dense Index	Sparse Index
Index entries	One entry for every search-key value (every record, if the key is unique)	One entry only for some records — typically one per block
Memory / storage usage	Higher — index size grows proportionally with the number of records	Lower — index size grows proportionally with the number of blocks, which is far smaller
Applicability	Can be built on any file, ordered or unordered	Requires the data file to be physically sorted on the index key
Access speed (point query)	Slightly faster to confirm existence — index directly names the record, no extra scanning	Slightly slower — after locating the block via the index, a short sequential scan within the block is needed
Update overhead	Higher — every insert/delete must update the index entry for that exact record	Lower — only insertions/deletions that affect a block's first record change the index

Analysis

Dense indexing offers marginally faster confirmation of a key's existence and works on unordered files, but at the cost of significantly higher memory usage since every record needs an entry. Sparse indexing is far more memory-efficient — proportional to block count rather than record count — but requires the file to be sorted, and each lookup needs a short within-block scan after the index step. In practice, sparse indexes are preferred for large, ordered files to save space, while dense indexes are used for secondary access paths or files that cannot be kept in sorted order.

Q8. Indexed Sequential File Organization vs Direct (Hashed) File Organization

Aspect	Indexed Sequential	Direct (Hashed) File Organization
Basis of access	Data physically sorted, with a multi-level index guiding search	Address computed directly from the key via a hash function

Aspect	Indexed Sequential	Direct (Hashed) File Organization
Exact-match search	Fast — $O(\log n)$ via index descent	Very fast — $O(1)$ average via direct hash computation
Range queries	Well supported — sorted order allows efficient sequential scans after the index locates the start	Poorly supported — hashing destroys key order, so ranges need a full scan
Insertion / deletion	Needs overflow-area management or periodic reorganization to preserve order	Generally fast ($O(1)$ average) but degrades with collisions/overflow as load factor rises
Storage efficiency	Reasonably efficient; index adds modest overhead	Deliberately kept below 100% load factor (e.g., 70–80%) to limit collisions, so some space is reserved

Advantages and limitations

- Indexed sequential — advantage: supports both fast exact-match and fast range queries; limitation: insertions can be costly and the file periodically needs reorganization to keep the overflow area small.
- Direct (hashed) — advantage: fastest possible average-case exact-match search and insertion; limitation: cannot efficiently support range queries or ordered retrieval, and performance degrades as collisions increase.

Choice depends on the query mix: indexed sequential organization suits applications needing both exact and range access (e.g., banking statements), while direct/hashed organization suits applications dominated by pure key lookups with little need for ordering (e.g., session/token stores).

Q9. Single-Level Indexing vs Multi-Level Indexing

Aspect	Single-Level Indexing	Multi-Level Indexing
Structure	One flat index mapping keys directly to data block addresses	A hierarchy of indexes — an index over the index — continuing until the top level fits in one block
Suitability for large files	Poor — once the index itself grows too large to fit in memory/one block, searching the index becomes slow (linear or binary search over many blocks)	Good — even a huge index is reduced to a small number of block accesses by descending levels (this is exactly the structure a B+ Tree implements)
Search cost	$O(\log b)$ with binary search on the index, but b (index blocks) can itself be large for huge files	$O(\log_p n)$, where p is the fan-out per level — height stays very small (often 3–4) even for millions of records
Maintenance on insert/delete	Simple structure but a large index still shifts significantly on updates	More complex to maintain (splits/merges may cascade across levels) but changes remain localized and logarithmic

Analysis

Single-level indexing is adequate only while the index itself is small enough to search quickly (ideally memory-resident). As the data file — and therefore the index — grows into the millions of records, a single-level index no longer fits conveniently in memory, and searching it becomes a bottleneck. Multi-level indexing solves this by recursively indexing the index, so that even a very large dataset is reduced to a handful of logarithmic-cost steps. This is precisely why real-world DBMSs use multi-level, tree-structured (B+ Tree) indexes rather than flat single-level indexes for large tables.

Q10. Evaluating Hashing vs Indexing for Exact-Match and Range Queries

Query type	Hashing (Static/Dynamic)	Indexing (B+ Tree, dense/sparse)
Exact-match query	Excellent — $O(1)$ average; the hash function computes the bucket address directly	Good — $O(\log n)$; must descend the tree/index levels, slower than hashing but still efficient
Range query	Very poor — hashing scatters keys, so a range scan degenerates to $O(n)$, a full file scan	Excellent — $O(\log n + k)$; index descent to the range start, then a fast sequential scan of k matching records via ordered structure/linked leaves
Growth handling	Static: degrades and needs reorganization; Dynamic (extendible/linear): scales incrementally	B+ Tree self-balances automatically; height grows logarithmically with data size
Typical use case	Point lookups — e.g., session tokens, primary-key equality lookups, caching layers	Ordered/range access — e.g., date-range reports, "between" queries, sorted listings, joins

Evaluation

The two techniques are complementary rather than strictly comparable on a single scale: hashing dominates for pure exact-match workloads, delivering constant-time average performance that no tree-based index can beat, but it is fundamentally unsuited to range queries because it deliberately destroys key ordering to spread values uniformly across buckets. Indexing (particularly B+ Tree-based) sacrifices a small amount of exact-match performance ($O(\log n)$ instead of $O(1)$) but gains the ability to answer range queries efficiently, since the ordered/linked structure allows a scan to proceed directly from the first qualifying key.

Overall verdict: for a workload dominated by exact-match lookups on a single key (e.g., authentication, caching), hashing is the superior technique. For a workload that requires range queries, sorting, or is likely to combine both patterns (the common case in real database systems), indexing — specifically a B+ Tree — is the more robust overall choice, since it performs acceptably on exact matches while excelling at ranges. This is why production DBMSs default to

B+ Tree indexes for general-purpose tables and reserve hash indexes for specialized, equality-only access paths (e.g., PostgreSQL's HASH index type, or in-memory hash-based key-value stores).